

YOLO

Clase práctica 20

Motivación

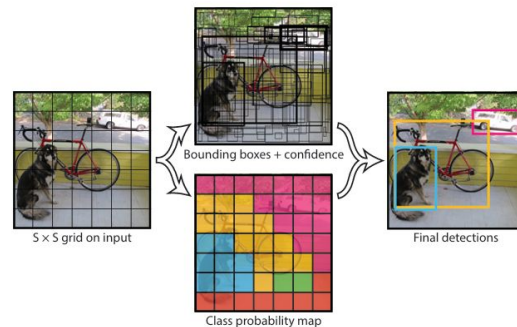
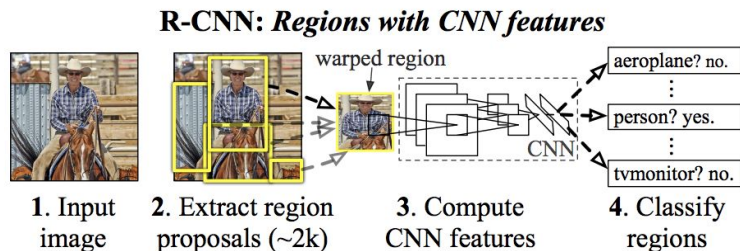
- ¿Cómo funciona **YOLO**?
- ¿De qué manera podemos usarla en el **TP**?
- ¿Cómo se ve un ejemplo de implementación?

You Only Look Once

Replantea el problema de “object detection” en uno de una **única regresión**. De píxeles a coordenadas de cajas y probabilidad de clases.

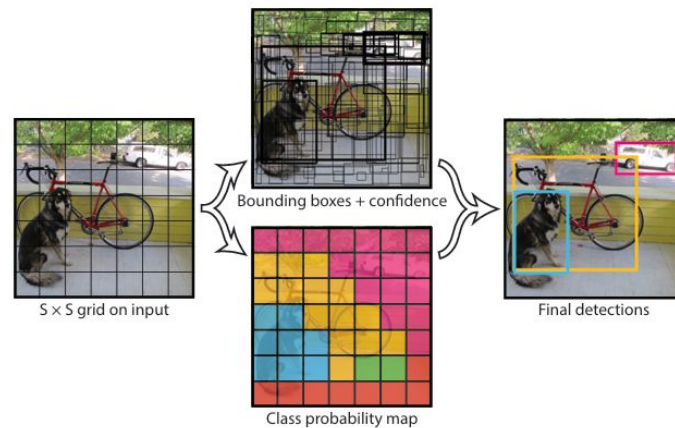
Antes proponíamos regiones, las clasificábamos, ajustábamos las cajas e intentábamos que no se pisen o estén duplicadas.

Ahora es todo en un simple paso de una red neuronal.



Bueno, pero cómo funciona?

1. Se divide la imagen en un **grilla de $S \times S$** .
2. Cada celda va a ser responsable de detectar un objeto **si el centro del objeto cae dentro**.
3. Cada celda predice:
 - a. B Bounding Boxes y su confianza
 - b. C probabilidades de clase condicionales en objeto.
4. En inferencia se combinan estas predicciones
5. Se eliminan las predicciones débiles y las cajas duplicadas con **NMS**.



NMS? Non-Maximum Suppression

Es una forma de resolver cajas con el mismo objeto cercanas.

1. Ordena cajas por confianza
2. Se queda con la caja de mayor score
3. Calcula el IoU con las demás
4. Elimina las muy solapadas
5. Repito sobre candidatas restantes en otras zonas si quedan.

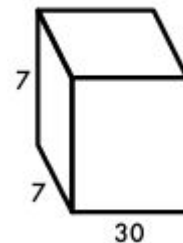
Zoom en el Ítem 3

1. Para cada Bounding Box predecimos
 - a. x, y = centro de la caja relativo a la celda
 - b. w, h = ancho y alto relativos a la imagen.
 - c. $\text{confianza} = \text{Prob}(\text{hay Objeto}) \times \text{IoU}(\text{pred}, \text{truth})$
2. Para cada celda una $\text{Prob}(\text{Clase } i \text{ condicionado a que haya objeto})$. En test queda:

$$\text{Pr}(\text{Class}_i | \text{Object}) * \text{Pr}(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \text{Pr}(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}} \quad (1)$$

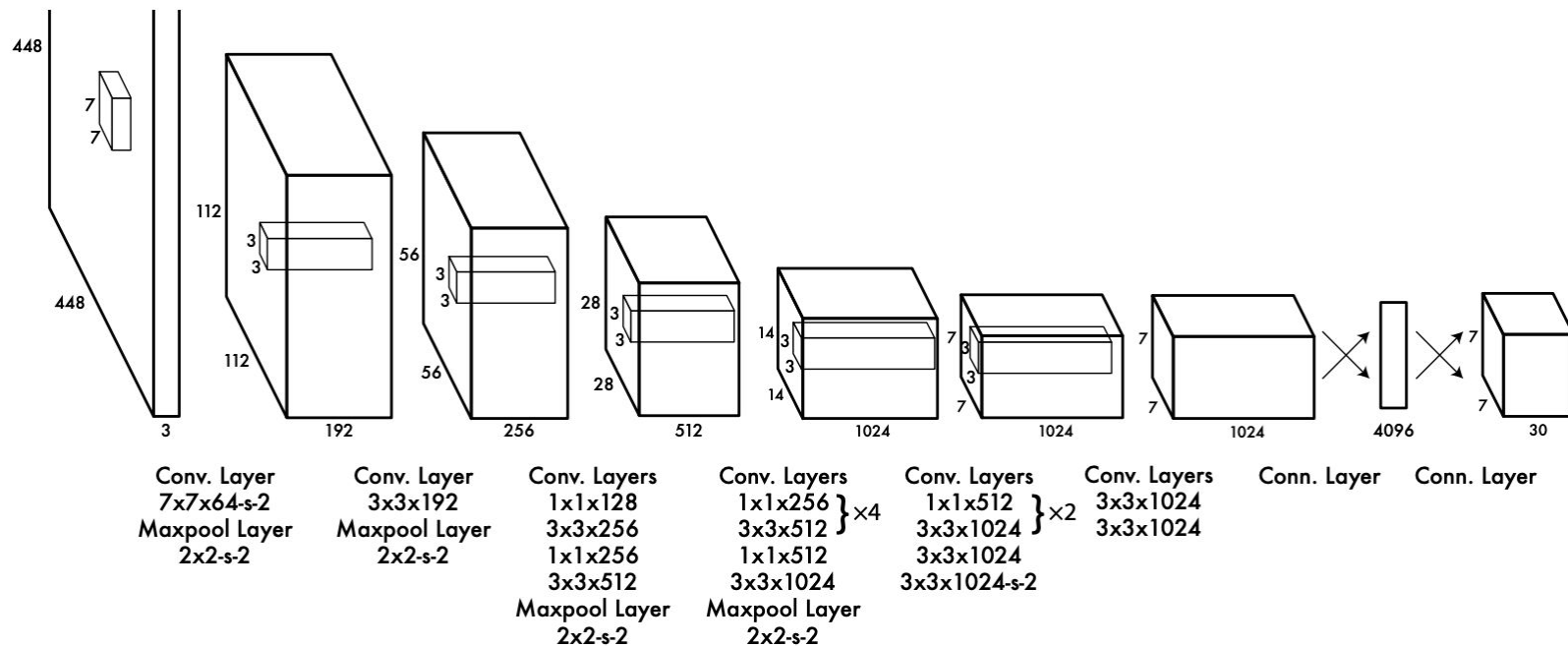
Entonces, cómo quedan nuestras predicciones?

Tengo **S x S celdas, B bounding boxes y C clases**. Además cada bounding box tiene (x,y,w,h,c). Usemos como ejemplo $S = 7$, $B = 2$, $C = 20$. El output lo podemos ver como un tensor donde es $S \times S \times (B \times 5 + C)$



Entonces para cada celda, en este caso, 49 (7×7), tengo la información necesaria de localización, confianza y clase.

Arquitectura



Cómo las evaluo?

loss function:

$$\begin{aligned} & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \end{aligned}$$

Penalizo las coordenadas de las BB.

Penalizo detectó falso o si no detectó

Penalizo clases incorrectas

Conclusiones de YOLO

Es rápido al plantear todo en una única regresión.

Entrenable end to end

Disponible en tiempo real

Errores de localización y problemas si hay objetos muy chicos o cercanos entre si dado que el tamaño de la celda es una limitación y centros de objetos cercanos también complican la asignación.

YOLO no va por la detección fina en principio, su principal contribución cuando se publicó fue replantear el problema de detección.

Taller Segmentación + Object Detection

Primero veamos cómo podemos usar **YOLO**:

https://colab.research.google.com/drive/1XBiOZ8M_27kDRgG-xfAwdTHF0-elGg5d?usp=sharing

Vamos a **fine-tune** para un dataset particular:

https://colab.research.google.com/drive/1CLX1j6JaXxw0kMershwyZY_NwOTvWh_A?usp=sharing